

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO FACULTAD DE
INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES



Manual de Calidad (SQAP)

"iNeon"

Integrantes:

Nro.	Nombre	Registro
1	Rodriguez Guzman Paul Jherson	218046235
2	Zenteno Rojas Edmundo Junior	221046461

Materia: Ingeniería de Software II INF512-SB

Docente: Msc. Ing. Martínez Canedo Rolando Antonio

Grupo: 7

Santa cruz – Bolivia

1. INTRODUCCIÓN	4
1.1 ANTECEDENTES.....	4
1.2 OBJETIVOS	4
1.2.1 OBJETIVO GENERAL	4
1.2.2 OBJETIVOS ESPECÍFICOS	4
1.3 MISIÓN.....	5
1.4 VISIÓN	5
1.5 POLÍTICAS DE CALIDAD.....	5
1.6 SLOGAN	6
2. PLAN DE ASEGURAMIENTO DE CALIDAD DE SOFTWARE (SQAP)	6
2.1 PROPÓSITO	6
2.1.1 OBJETIVO	6
2.1.2 DESCRIPCIÓN	6
2.1.3 ALCANCE	7
2.2 DOCUMENTOS DE REFERENCIA	8
2.3 GESTIÓN	8
2.3.1 ORGANIZACIÓN.....	8
2.3.2 TAREAS	9
2.3.3 ROLES Y RESPONSABILIDADES.....	10
2.3.3.1. RESPONSABILIDADES DEL GRUPO DE DESARROLLO.....	10
2.3.3.2. RESPONSABILIDADES DEL CLIENTE.....	10
2.3.3.3. RESPONSABILIDAD DE LA SQA	11
2.4 DOCUMENTACIÓN.....	12
2.4.1 Especificación de Requisitos de Software	12
2.4.2 Descripción del Diseño del Software (SDD)	14
2.4.3 Plan de Verificación y Validación	17
2.5 ESTÁNDARES, PRÁCTICAS Y CONVENCIONES.....	27
2.5.1 ESTÁNDAR DE CODIFICACIÓN	27
2.5.2 ESTÁNDAR DE COMENTARIOS.....	28

2.5.3 RESPONSABLES DE VERIFICAR EL CUMPLIMIENTO	29
2.6 REVISIONES Y AUDITORÍAS	30
Evaluación de la calidad de los productos.....	30
Revisar el ajuste al proceso	31
Revisión Técnica Formal (RTF)	31
2.7 GESTIÓN DE CONFIGURACIÓN	34
2.8 GESTIÓN DE PROBLEMAS Y ACCIONES CORRELATIVAS	35
2.9 HERRAMIENTAS, TÉCNICAS Y METODOLOGÍAS	37
2.10 CONTROL DE CÓDIGO	39
2.11 CONTROL DE MEDIOS	40
2.12 CONTROL DE SUMINISTROS Y SUBCONTRATOS	42
2.13 RECOLECCIÓN, MANTENIMIENTO Y RETENCIÓN DE REGISTROS	43

1. INTRODUCCIÓN

1.1 ANTECEDENTES

iNeon es una empresa mediana dedicada al desarrollo de software a medida, que ha decidido formalizar sus procesos de calidad para asegurar productos de **alta calidad** a sus clientes. En un mercado competitivo, la satisfacción del cliente y la confiabilidad del software son fundamentales. Por ello, iNeon adopta un **Plan Institucional de Aseguramiento de la Calidad del Software (SQAP)** basado en el estándar **IEEE 730**, con el fin de establecer lineamientos claros y homogéneos de calidad en todos sus proyectos de desarrollo. Este documento recoge las **mejores prácticas** de la industria y las adapta a la realidad de iNeon, garantizando que cada proyecto siga procesos de calidad consistentes y alineados con estándares internacionales.

1.2 OBJETIVOS

1.2.1 OBJETIVO GENERAL

Implementar un **marco institucional de aseguramiento de la calidad del software** que garantice que todos los productos de software desarrollados por iNeon cumplan con los requisitos acordados y estándares de calidad establecidos, aumentando la **satisfacción del cliente** y la **eficiencia** de los procesos internos.

1.2.2 OBJETIVOS ESPECÍFICOS

- **Estandarizar procesos de calidad:** Definir procedimientos comunes de verificación y validación aplicables a todos los proyectos, asegurando consistencia en la forma de trabajar.
- **Cumplimiento de estándares:** Adoptar estándares reconocidos para codificación, documentación, pruebas y gestión, y verificar su cumplimiento en cada proyecto.
- **Detección temprana de defectos:** Institucionalizar revisiones técnicas formales y auditorías para identificar y corregir defectos en fases tempranas del ciclo de vida, reduciendo costos y retrabajos posteriores.
- **Mejora continua:** Establecer mecanismos de medición de la calidad (métricas, registros de incidencias, retroalimentación) que permitan retroalimentar el proceso y mejorar continuamente el desempeño de la empresa en materia de calidad.

- **Formación y conciencia en calidad:** Asegurar que todo el personal de iNeon reciba capacitación en las políticas, estándares y prácticas de calidad definidas.

1.3 MISIÓN

La misión de iNeon es brindar soluciones de software a medida de *alta calidad* que satisfagan plenamente las necesidades de nuestros clientes. Logramos esto mediante la excelencia técnica, la innovación constante y un fuerte compromiso con la calidad y la mejora continua en todos nuestros procesos. Cada proyecto se aborda con profesionalismo y dedicación, garantizando que el resultado final aporte valor tangible al cliente y cumpla con los más altos estándares de la industria.

1.4 VISIÓN

La visión de iNeon es ser reconocida internacionalmente como una empresa líder en desarrollo de software a medida, destacada por su compromiso con la calidad, la confiabilidad de sus productos y la satisfacción del cliente. Aspiramos a construir una reputación donde cada solución entregada por iNeon represente innovación, robustez y confianza. Nos proyectamos a futuro como referentes en metodologías de ingeniería de software de calidad, contribuyendo al progreso tecnológico y estableciendo relaciones de largo plazo con nuestros clientes basadas en la excelencia.

1.5 POLÍTICAS DE CALIDAD

- **Enfoque en el cliente:** Los requisitos y expectativas del cliente guían todo nuestro trabajo. Nos comprometemos a entregar software que cumpla y exceda dichos requisitos, asegurando su satisfacción.
- **Cumplimiento de estándares y normativa:** Desarrollamos nuestros proyectos conforme a estándares reconocidos de ingeniería de software (por ejemplo, IEEE 730 para planes de calidad, ISO/IEC 25010 para calidad de producto, ISO 9001 para gestión de la calidad). Esto garantiza un lenguaje común y prácticas probadas en todos nuestros procesos.
- **Prevención sobre corrección:** Priorizamos la prevención de defectos mediante revisiones sistemáticas, pruebas exhaustivas y cumplimiento de procesos, en lugar de depender únicamente de correcciones posteriores. Identificar problemas en etapas tempranas es clave para la calidad y la eficiencia.
- **Mejora continua:** Establecemos objetivos de calidad y medimos regularmente nuestro desempeño. Toda desviación o área de mejora detectada se analiza y se

traducen en acciones correctivas o preventivas. Fomentamos una cultura donde el personal propone mejoras a procesos, herramientas y productos.

- **Participación de todos:** La calidad es responsabilidad de cada miembro de iNeon. Se espera que todos sigan los procesos definidos, utilicen las herramientas de calidad provistas y reporten oportunamente cualquier incidencia o riesgo que pueda afectar la calidad del producto.

1.6 SLOGAN

“Innovación y calidad en cada solución de software.”

2. PLAN DE ASEGURAMIENTO DE CALIDAD DE SOFTWARE (SQAP)

2.1 PROPÓSITO

Especificar las actividades que se realizarán para asegurar la calidad del software desarrollado por iNeon. En este documento se detallan los productos que serán objeto de revisión y los estándares, normas y métodos que se aplicarán; asimismo se describen los procedimientos que se utilizarán para verificar que la elaboración de cada producto se realiza de acuerdo con el modelo de ciclo de vida adoptado; y se establecen los mecanismos para informar a los responsables de cada producto sobre los defectos encontrados, realizando seguimiento hasta sus correcciones.

2.1.1 OBJETIVO

Definir un conjunto de normas, prácticas y actividades destinadas a garantizar la calidad en el desarrollo de software. Esto incluye establecer criterios de calidad claros, designar procesos de verificación y validación apropiados, y asignar responsabilidades para ejecutar dichos procesos.

2.1.2 DESCRIPCIÓN

Calidad de software es el grado en que el software cumple con los requisitos explícitamente establecidos, se ajusta a los estándares de desarrollo aplicados y satisface los requisitos implícitos que todo usuario espera de un producto profesional.

Esto implica que un software de calidad debe ser:

- **Correcto y completo** respecto a los requisitos funcionales.
- **Fiable y eficiente** en su desempeño.
- **Usable y mantenible**, facilitando su operación y evolución.
- **Seguro e íntegro**, protegiendo los datos y funcionando sin errores críticos.
- **Portátil y adaptable** en distintos entornos, si aplica.

A través de la implementación de este SQAP institucional, iNeon pretende asegurarse de que cada proyecto incorpore estos atributos de calidad durante su ciclo de vida.

2.1.3 ALCANCE

El presente plan se aplica a todos los proyectos de desarrollo de software emprendidos por iNeon, independientemente de su tamaño o complejidad. Proporciona enfoques y directrices a seguir para asegurar al cliente la calidad deseada en cada entrega. El alcance del SQAP cubre todas las actividades involucradas en el proceso de desarrollo de software de la empresa, desde la fase de obtención de requisitos hasta la entrega y despliegue del producto, incluyendo actividades de verificación y validación en cada etapa.

El enfoque adoptado para estructurar el ciclo de vida del software se basa en el **Marco de Trabajo Genérico del Proceso de Software (MTPS)**, que comprende las siguientes cinco fases fundamentales:

- **Comunicación:** Se establece un entendimiento común con el cliente sobre los requisitos y expectativas del software.
- **Planeación:** Se definen recursos, cronograma, estimaciones, riesgos y estrategias de gestión de calidad del proyecto.
- **Modelado:** Incluye la ingeniería de requisitos y el diseño del software, considerando funcionalidad, arquitectura y estructuras de datos.
- **Construcción:** Comprende la codificación, pruebas unitarias e integración, con base en los estándares de calidad definidos por iNeon.
- **Despliegue:** Involucra la entrega, instalación y soporte inicial del producto al cliente, junto con la recolección de retroalimentación.

Estas fases serán gestionadas bajo un marco institucional de aseguramiento de la calidad conforme a los lineamientos del estándar IEEE-730, con mecanismos de evaluación continua y mejora del proceso.

2.2 DOCUMENTOS DE REFERENCIA

En esta sección se listan los documentos y estándares de referencia que sirven de base o complemento a este Plan de Aseguramiento de la Calidad del Software. Estos documentos contienen requerimientos, lineamientos o información relevante que el SQAP toma en consideración para definir las actividades de calidad:

- **IEEE Std 730-2014** – *Standard for Software Quality Assurance Processes*. Estándar IEEE que proporciona los requisitos para la preparación y el contenido de planes de aseguramiento de la calidad del software (SQAP).
- **IEEE Std 1012-2016** – *Standard for System and Software Verification and Validation*. Estándar IEEE para procesos de verificación y validación de software, utilizado como guía para elaborar el plan de V&V .
- **IEEE Std 1028-2008** – *Standard for Software Reviews and Audits*. Estándar que describe procedimientos formales de revisiones (inspecciones, walkthroughs, etc.) y auditorías de software, empleado como base para las prácticas.
- **IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018** – *Especificación de Requisitos de Software*. Estándar de IEEE (sucesor: ISO 29148) que define el formato y contenidos recomendados de una Especificación de Requisitos de Software (SRS).
- **IEEE Std 1016-2009** – *Recommended Practice for Software Design Descriptions*. Recomendación IEEE para la documentación de diseño de software (SDD).
- **Plan de Proyecto y Plan de Configuración (SCMP) de iNeon**: Documentos internos que describen la planificación general del proyecto y las prácticas de gestión de configuración respectivamente.
- **Manual de Calidad y Procedimientos Internos de iNeon**: Documentación corporativa que recoge la política de calidad.

2.3 GESTIÓN

2.3.1 ORGANIZACIÓN

La responsabilidad de la calidad del software en iNeon recae en una estructura organizativa definida que involucra tanto al **equipo de desarrollo** de cada proyecto como a un **grupo independiente de Aseguramiento de Calidad (SQA)** que supervisa y audita el cumplimiento del proceso. A nivel institucional, existe un **Gerente de Calidad de Software** o líder del grupo SQA que reporta a la alta gerencia para garantizar la independencia y autoridad necesarias. Este gerente SQA es el encargado de establecer el SQAP, actualizarlo y velar por su implementación efectiva en todos los proyectos.

Cada proyecto de desarrollo cuenta con un **Gerente de Proyecto** (Project Manager) responsable de la ejecución global del proyecto, incluyendo la implementación de las actividades SQA en el plan del proyecto. El Gerente de Proyecto colabora con el grupo SQA para planificar **hitos de calidad** (revisiones, puntos de control) dentro del cronograma.

El **Grupo SQA** institucional está conformado por ingenieros o auditores de calidad capacitados en procesos de software. Puede estar organizado de forma *matricial*, asignando uno o varios representantes SQA a cada proyecto. Estos representantes actúan como **auditores internos** y facilitadores de la calidad: supervisan que se sigan los procesos, realizan revisiones independientes y asesoran al equipo de desarrollo en prácticas de mejora de calidad.

Diagrama organizacional:

- **Gerencia General / Director de Operaciones**
 - └ **Gerente de Calidad de Software (SQA Manager)**
 - └ **Equipo/Grupo SQA Institucional** (auditores de calidad, ingenieros SQA)
 - └ *Representantes SQA asignados a cada Proyecto*
- **Gerentes de Proyecto** (en cada proyecto, colaboran con SQA)
 - └ **Equipo de Desarrollo** (analistas, diseñadores, desarrolladores, testers del proyecto)

2.3.2 TAREAS

La gestión del aseguramiento de calidad implica planificar, coordinar y monitorear diversas *tareas SQA* a lo largo del ciclo de vida. Algunas de las tareas principales son:

- **Planificación de la calidad:** El grupo SQA participa en la planificación inicial del proyecto para integrar las actividades de calidad en el cronograma (por ejemplo, programar revisiones técnicas formales después de concluir el SRS y el diseño, planificar pruebas, etc.).
- **Difusión de procesos y estándares:** SQA se asegura de que el equipo de desarrollo conozca los **procesos aprobados**, estándares y procedimientos que deben seguir.
- **Supervisión continua:** A lo largo del desarrollo, SQA **supervisa** las actividades de ingeniería de software para verificar su ajuste al proceso definido. Esto significa observar y auditar cómo se llevan a cabo tareas como levantamiento de requisitos,

diseño, codificación y pruebas, comprobando que se sigan las metodologías y estándares acordados.

- **Revisiones y auditorías:** SQA organiza y lleva a cabo diversas revisiones de productos de trabajo (documentos, código, casos de prueba) y auditorías de proceso.
- **Gestión de no conformidades:** Cuando SQA detecta desviaciones o problemas (documentación incompleta, código que no sigue estándares, requisitos ambiguos, etc.), documenta dichas incidencias conforme al procedimiento.
- **Reporte de estado de calidad:** SQA prepara informes periódicos para la gerencia de proyecto y alta dirección sobre el *estado de la calidad* en el proyecto. Estos informes pueden incluir métricas de defectos, resultados de revisiones, riesgos de calidad detectados y recomendaciones.
- **Coordinación con Gestión de Configuración:** La actividad SQA también verifica que se apliquen adecuadamente los procedimientos de control de versiones y cambios.
- **Certificación final del producto:** Antes de la entrega al cliente, SQA realiza una **evaluación final de calidad** (Quality Gate) para certificar que el software cumple con todos los criterios acordados (resultados de pruebas aceptables, documentación completa, requisitos trazados, etc.).

2.3.3 ROLES Y RESPONSABILIDADES

2.3.3.1. RESPONSABILIDADES DEL GRUPO DE DESARROLLO.

- Desarrollar un producto de software en base a lo de finido en el SQAP y los contratos establecidos con el cliente.
- Generar la debida documentación definida en la SQAP acerca de cada una de sus actividades con el fin de llevar un control de las mismas.
- Entregar la documentación de desarrollo que se exige en el plan (SQAP).

2.3.3.2. RESPONSABILIDADES DEL CLIENTE.

- Proveer la información necesaria para el desarrollo del software con el fin de satisfacer sus necesidades.
- Brindar los recursos y condiciones necesarias para elaborar el software.

- Participar activamente en la organización del SQA para obtener óptimos resultados.

2.3.3.3. RESPONSABILIDAD DE LA SQA

- Establecimiento del plan SQA para el proyecto.
- Participar en el desarrollo de la descripción del proceso de software.
- Revisión de las actividades de ingeniería del software para verificar su ajuste al proceso del software.
- Auditoria de los productos de software designados para verificar el ajuste con los definidos como parte del proceso de software.
- Asegurar que las desviaciones del trabajo y los productos del software se documentan y se manejan de acuerdo con un procedimiento establecido.
- Registrar lo que no se ajuste a los requisitos e informar a sus superiores.
- Coordinar el control y la gestión de cambios.
- Analizar las métricas del software.

Esta organización es responsable de:

- Garantizar la calidad del producto de software desarrollado.
- Implantar normas y actividades para el desarrollo del software.
- Realizar reuniones para resolver los posibles conflictos durante el desarrollo del software.
- Aprobar y publicar el SQAP.
- Observar las deficiencias en el SQAP.
- Mejorar el SQAP, recomendando modificaciones o correcciones con el fin de obtener resultados óptimos.

- Autorizar la implantación del software.
- Enfoque de gestión de calidad.
- Tecnologías (métodos y herramientas).
- Revisiones Técnicas Formales.
- Estrategia de pruebas.
- Control de la documentación y de cambios.
- Procedimientos que aseguren ajustes a los estándares.
- Mecanismos de medición y generación de informes.

2.4 DOCUMENTACIÓN

La calidad del software no solo depende del código, sino también de la **documentación** que guía y registra el proceso de desarrollo. Una parte esencial del SQAP es asegurar que ciertos documentos clave sean elaborados, revisados y gestionados correctamente. Estos documentos sirven como base para entender el producto, planificar pruebas y mantener la trazabilidad de los requisitos a lo largo del proyecto. A continuación, se describen los principales artefactos documentales considerados y cómo encajan en el aseguramiento de calidad:

2.4.1 Especificación de Requisitos de Software

Esta documentación es elaborada por el desarrollador, y se basa en el estándar ANSI /

IEEE-Std 830 “Guía para especificaciones de requerimientos de software” (**SRS**).

La SRS deberá describir claramente y de forma precisa cada uno de los requerimientos del Software, tal como: funciones, rendimiento, restricciones de diseño y atributos.

MODELO A USAR PARA EL CONTENIDO DEL SRS

1. INTRODUCCION

1.5 Objetivo

2.5 Alcance

3.5 Definiciones, acrónimos y abreviaciones

4.5 Referencias

5.5 Revisión

2. DESCRIPCION GENERAL

2.1 Perspectiva del producto

2.2 Funciones del producto

2.3 Características de los usuarios

2.4 Restricciones generales

2.5 Asunciones y dependencias

3. ESPECIFICACION DE REQUERIMIENTOS

3.1 Requerimiento Funcional

3.1.1. Introducción

3.1.2. Entradas

3.1.3. Procesos

3.1.4. Salidas

3.1.5. Interfaces externas

3.1.5.1. Interfaces del usuario

3.1.5.2. Interfaces del hardware

3.1.5.3. Interfaces del software

3.1.6 Requerimientos de rendimiento

3.1.7 Representación del diseño

3.1.8 Cumplimientos con estándares

3.1.9 Limitaciones del hardware

3.1.10 Atributos

3.1.10.1. Disponibilidad

3.1.10.2. Seguridad

3.1.10.3. Mantenibilidad

3.1.10.4. Transferencia / Conversión

3.1.10.5 Prevenciones

3.1.11 Otros requerimientos

3.1.11.1. Base de datos

3.1.11.2. Operaciones

3.1.11.3. Adaptaciones

APENDICES

INDICE

ANEXOS

2.4.2 Descripción del Diseño del Software (SDD)

La generación y documentación de la descripción del diseño de Software se basa en el estándar ANSI / IEEE – Std 1016 “RECOMMENDED PRACTICE FOR SOFTWARE DESCRIPTIONS”

Organización de la SDD dentro de Vistas de Diseño

VISTA DE DISEÑO	ALCANCE	ATRIBUTOS DE ENTIDAD	EJEMPLOS DE REPRESENTACIONES
Descripción de descomposición	Partición del sistema dentro de entidades de diseño.	Identificación, tipo, objetivo, función, subordinación.	Diagrama de descomposición, jerarquía y lenguaje natural.
Descripción de dependencia	Descripción de las relaciones entre entidades y recursos del sistema.	Identificación, tipo, objetivo, dependencias, recursos.	Diagrama de estructura.
Descripción de interfaces	Lista de cada interfaz de diseñador, programador, o pruebas necesarias para conocer el uso de la entidad de diseño que componen el sistema.	Identificación, función, interfaces.	Tablas de parámetros.
Descripción de detalle	Descripción de los detalles de diseño internos en una entidad.	Identificación, procesamiento, datos.	Diagrama de flujos.

MODELO A USAR PARA EL CONTENIDO DEL SDD

1. INTRODUCCION

- o Objetivo
- o Alcance

- o Definiciones, acrónimos y abreviaciones

2. REFERENCIAS

3. DESCRIPCION DE DESCOMPOSICION

3.1. Descomposición de módulo

3.1.1. Descripción del módulo 1

3.1.2. Descripción del módulo 2

3.1.n. Descripción del módulo n

3.2. Descomposición de procesos concurrentes

3.2.1. Descripción del proceso 1

3.2.2. Descripción del proceso 2

3.2.n. Descripción del proceso n

3.3. Descomposición de datos

3.3.1. Descripción de la entidad de datos 1

3.3.2. Descripción de la entidad de datos 2

3.3.n. Descripción de la entidad de datos n

4. DESCRIPCION DE DEPENDENCIA

- o Dependencia entre módulos
- o Dependencia entre procesos
- o Dependencia entre datos

5. DESCRIPCION DE INTERFACES

5.1 Interfaces de módulo

5.1.1. Descripción del módulo 1

5.1.2. Descripción del módulo 2

5.1.n. Descripción del módulo n

5.2 Interfaces de procesos

5.2.1. Descripción del proceso 1

5.2.2. Descripción del proceso 2

5.2.n. Descripción del proceso n

6. DISEÑO DETALLADO

6.1. Diseño detallado del módulo

6.1.1. Detalle del módulo 1

6.1.2. Detalle del módulo 2

6.1.n. Detalle del módulo n

6.2. Diseño detallado de datos

6.2.1. Detalle de entidad de datos 1

6.2.2. Detalle de entidad de datos 2

6.2.n. Detalle de entidad de datos n

APENDICES

INDICE

ANEXOS

2.4.3 Plan de Verificación y Validación

La generación y documentación de la descripción del Plan de Verificación y Validación (SWP) es la siguiente:

MODELO A USAR PARA EL CONTENIDO DEL SWP

1. OBJETIVO
2. ALCANCE
3. DEFINICIONES, ACRONIMOS Y ABREVIACIONES
4. ORGANIZACIÓN RESPONSABLES
5. ESTRATEGIA DE VERIFICACION.

6. RECURSOS.

7. ENTREGABLES.

APENDICE

INDICE

La organización responsable por las tareas de verificación y validación del software es la organización de SQA comandada por la organización del consultor, la cual interactúa con la organización de desarrollo para alcanzar los objetivos del plan. En casos necesarios de conflictos extremos entre el consultor y el desarrollador se recurrirá al cliente.

5. Estrategia de Verificación

Esta sección presenta el enfoque recomendado para la verificación. Describe cómo se verificarán los elementos. Para cada tipo de prueba, se proporciona una descripción de la prueba y porque será implementada y ejecutada. Si un tipo de prueba no será implementada y ejecutada, se indicará brevemente cual es la prueba que no se implementará o ejecutará y una justificación de ello. Se indicarán las técnicas usadas y el criterio para saber cuando una prueba se completó (criterio de aceptación). Las pruebas se deben ejecutar usando bases de datos conocidas y controladas en un ambiente seguro.

5.1 Tipos de pruebas

5.1.1 Prueba de integridad de los datos y la base de datos

Justificación No se realizará este tipo de prueba debido a que la base de datos del cliente y la base de datos que usa el servidor intermedio(WSServer) solo admiten consultas de lectura por el momento. En caso que el cliente proponga el requisito de que un usuario pueda registrarse se tendrá en cuenta este tipo de prueba.

5.1.2 Prueba de Funcionalidad

Descripción La prueba de funcionalidad se enfoca en requerimientos para verificar que se corresponden directamente a casos de usos o funciones. Este tipo de prueba se basa en técnicas de caja negra, que consisten en verificar la aplicación y sus procesos interactuando con la aplicación por medio de la interfase de usuario y analizar los resultados obtenidos.

Justificación Este tipo de prueba es necesario para garantizar que el software hace lo que debe y sobre todo, lo que se ha especificado en el documento de requisitos.

Objetivo de la prueba Asegurar la funcionalidad apropiada del objeto de prueba, incluyendo la navegación, entrada de datos, proceso y recuperación.

Técnica Se usarán los casos de pruebas diseñados a partir de los casos de uso, luego se ingresan los datos de pruebas (válidos e inválidos) en la interfaz del dispositivo mientras se ejecuta la herramienta Appium o Selenium, el cual mediante el inspector se encargará de crear código para cada acción del tester. Si el software no hace lo que debe entonces se debe reportar el error al equipo de desarrollo y guardar el código generado para automatizar la prueba.

Criterio de aceptación Todas las pruebas planificadas se realizaron. Todos los defectos encontrados han sido debidamente identificados.

Consideraciones especiales Identificar o describir aquellos elementos o problemas (internos o externos) que impactaron en la implementación y ejecución de las pruebas de funcionalidad.

5.1.3 Prueba de Ciclo del Negocio

Descripción Esta prueba debe simular las actividades realizadas en el proyecto en el tiempo. Se debe identificar un período, que puede ser un año, y se deben ejecutar las transacciones y actividades que ocurrirían en el período de un año. Esto incluye todos los ciclos diarios, semanales y mensuales y eventos que son sensibles a la fecha.

Justificación Este tipo de prueba no se realizará, porque el sistema software a construir no tiene un ciclo de negocio definido en el documento de requisitos. Un ejemplo donde se podría ejecutar este tipo de prueba es en un sistema de administración (o contable), donde se genera una plantilla de pago cada año.

5.1.4 Prueba de Interfase de Usuario

Descripción Esta prueba verifica que la interfase de usuario proporcione al usuario el acceso y navegación a través de las funciones apropiada. Además asegura que los objetos presentes en la interfase de usuario se muestran como se espera y conforme a los estándares establecidos por la empresa o de la industria.

Justificación Se realizará este tipo de prueba debido a que la aplicación debe cumplir el requerimiento no funcional de usabilidad.

Objetivo de la prueba Verificar que: la aplicación sea de fácil uso y que la navegación a través de los elementos que se están probando reflejen los requerimientos, incluyendo manejo de ventanas, campos y métodos de acceso; los objetos de las ventanas y características, como menús, tamaño, posición, estado funcionen de acuerdo a los estándares mínimos definidos.

Técnica Para verificar la usabilidad se usará la técnica thinking aloud (pensando en voz alta). La técnica consiste en proporcionar a los usuarios el prototipo a probar y un conjunto de tareas a realizar.

- Se les pide que realicen las tareas y que expliquen en voz alta qué es lo que piensan al respecto mientras están trabajando con la interfaz, describiendo qué es lo que creen que está pasando, por qué toman una u otra acción o qué es lo que están intentando realizar. En definitiva, ¡cuantas más impresiones mejor!.

- Pensando en voz alta permite a los evaluadores comprender cómo el usuario se aproxima al objetivo con la interfaz propuesta y qué consideraciones tiene en la mente cuando la usa. El usuario puede expresar que la secuencia de etapas que le dicta el producto para realizar el objetivo de su tarea es diferente de la que esperaba.

Para verificar que se cumple el estándar, crear o modificar pruebas para cada ventana verificando la navegación y los estados de los objetos para cada ventana de la aplicación y cada objeto dentro de la ventana.

Criterio de aceptación La aplicación cumple con las métricas de usabilidad definida en el plan de calidad. Cada ventana ha sido verificada exitosamente siendo consistente con una versión de referencia o estándar establecido.

Consideraciones especiales Tener en cuenta la experiencia de usuario en cuanto a amigabilidad, intuitividad, baja curva de aprendizaje y facilidad de uso.

5.1.5 Prueba de Performance, Prueba de Carga, Prueba de Esfuerzo, Prueba de Volumen

Justificación No se realizarán este tipo de pruebas, debido a que la aplicación a construir no es un sistema que esté pensado (por la naturaleza del sistema) para manejar grandes volúmenes de tráfico.

5.1.6 Prueba de Seguridad y Control de Acceso

Descripción La Prueba de Seguridad y Control de Acceso se enfoca en dos áreas de seguridad: • Seguridad en el ámbito de aplicación, incluyendo el acceso a los datos y a las funciones de negocios. • Seguridad en el ámbito de sistema, incluyendo conexión, o acceso remoto al sistema.

La seguridad en el ámbito de aplicación asegura que, basado en la seguridad deseada los actores están restringidos a funciones o casos de uso específicos o limitados en los datos que están disponibles para ellos.

La seguridad en el ámbito de sistema asegura que, solo los usuarios con derecho a acceder al sistema son capaces de acceder a las aplicaciones y solo a través de los puntos de ingresos apropiados.

Justificación Se realizará este tipo de prueba debido a que la aplicación debe cumplir el requerimiento no funcional de seguridad.

Objetivo de la prueba Seguridad en el ámbito de aplicación: Verificar que un actor pueda acceder solo a las funciones o datos para los cuales su tipo de usuario tiene permiso.

Seguridad en el ámbito de sistema: Verificar que solo los actores con acceso al sistema y a las aplicaciones, puedan acceder a ellos.

Encriptación de datos sensibles: • Verificar métodos de encriptación y validar que los datos son enviados entre los componentes del sistema con el encriptado correcto.

Técnica Seguridad en el ámbito de aplicación: Identificar y hacer una lista de cada tipo de usuario y las funciones y datos sobre las que cada tipo tiene permiso.

Crear pruebas para cada tipo de usuario y verificar cada permiso creando operaciones específicas para cada tipo de usuario.

Modificar el tipo de usuario y volver a ejecutar las pruebas para los mismos usuarios. En cada caso, verificar que las funciones o datos adicionales están correctamente disponibles o son denegados.

Verificar métodos de encriptación y validar que los datos son enviados con el encriptado correcto

Acceso en el ámbito de sistema: Ver consideraciones especiales más abajo.

Criterio de aceptación Para cada tipo de actor conocido las funciones y datos apropiados están disponibles, y todas las operaciones funcionan como se espera y ejecutan las pruebas de Funcionalidad de la aplicación.

Consideraciones especiales El acceso al sistema debe ser discutido con el administrador del sistema o la red. Esta prueba no puede requerirse como tal, es una función del administrador del sistema o de la red.

5.1.7 Prueba de Fallas y Recuperación

Descripción La Prueba de Recuperación es un proceso en el cual la aplicación o sistema se expone a condiciones extremas, o condiciones simuladas, para causar falla, como fallas en dispositivos de Entrada/Salida o punteros a la base de datos inválidos. Los procedimientos de recuperación se invocan y la aplicación o sistema es monitoreado e inspeccionado para verificar que se recupera apropiadamente la aplicación o sistema y se logre la recuperación de datos.

Justificación A pesar de que no es un requisito no funcional se tendrá en cuenta para asegurar que el software puede recuperarse de fallas de hardware, software o mal funcionamiento de la red sin pérdida de datos o de integridad de los datos.

Objetivo de la prueba Verificar que los procesos de recuperación (manual o automáticos) recuperen apropiadamente la base de datos, aplicaciones y sistema a un estado conocido y deseado. En la prueba se incluyen los siguientes tipos de condiciones: • interrupción de energía al cliente • interrupción de energía al servidor • interrupción de comunicaciones mediante los servidores de la red • interrupción de comunicación o pérdida de energía de los discos del servidor o con los controladores • ciclos incompletos (procesos de filtrado de datos interrumpidos, procesos de sincronización de datos interrumpidos) • punteros a la base de datos o claves inválidos • elementos de datos en la base de datos inválidos o corruptos.

Técnica Se deben usar las pruebas creadas para probar Funcionalidad para crear una serie de operaciones. Una vez logrado el punto de comienzo deseado, se deben realizar o simular las siguientes acciones, individualmente: • Interrumpir la energía del cliente: apagar el PC. • Interrumpir la energía del servidor: simular o iniciar el proceso de apagado del servidor. • Interrupción por medio de los servidores de red: simular o iniciar la pérdida de comunicación con la red (desconectar físicamente la comunicación o apagar el servidor de red o router • Interrumpir la comunicación o quitar la energía de los discos del servidor o sus controladores: simular o eliminar físicamente la comunicación con uno o

más controladores de disco o los discos. • Una vez que se lograron o simularon estas condiciones, se deben invocar los procedimientos de recuperación. • Las pruebas de ciclos incompletos utilizan la misma técnica excepto que los procesos de bases de datos deben ser abortados a sí mismos o terminados prematuramente. • Las últimas dos pruebas requieren que se logre un estado conocido de la base de datos. Se deben corromper manualmente campos de la base de datos, punteros y claves trabajando directamente sobre la base de datos (utilizando herramientas para la base de datos). Se deben ejecutar las pruebas de Funcionalidad y Ciclo de negocio y verificar que los ciclos se completen.

Criterio de aceptación En todos los casos, la aplicación, la base de datos y el sistema deben, en la realización procedimientos de recuperación, volver a un estado conocido y deseable. Este estado incluye corrupción de datos limitada al los campos, punteros o claves corruptos conocidos, y reportes indicando los procesos u operaciones que no se completaron debido a las interrupciones.

Consideraciones especiales Los procedimientos para desconectar cables (simulando falta de energía o pérdida de comunicación) no son deseables o factibles. Se pueden requerir métodos alternativos, como software de diagnóstico. Se requieren los grupos de recursos de Sistemas, Bases de datos y Red.

Estas pruebas deben ejecutarse fuera del horario de trabajo normal o en una máquina aislada.

5.1.8 Prueba de Configuración

La Prueba de Configuración verifica el funcionamiento del software con diferentes configuraciones software y hardware.

Justificación Se realizará este tipo de prueba, debido a que la aplicación hecha en android nativo debe funcionar en distintos dispositivos que usen la misma versión del sistema operativo android.

Objetivo de la prueba Verificar que el software funcione apropiadamente en las configuraciones requeridas de hardware y software.

Técnica Usar las pruebas de Funcionalidad. • Abrir y cerrar varias sesiones de software que no son objeto de prueba, como parte de la prueba o antes de comenzar la prueba. • Ejecutar operaciones seleccionadas para simular la interacción del actor con el software objeto de prueba y con el software que no es objeto de prueba. • Repetir los

procedimientos anteriores minimizando la memoria convencional disponible en la máquina cliente.

Criterio de aceptación Por cada combinación de software objeto de prueba y software que no es objeto de prueba, todas las operaciones son completadas exitosamente sin fallas.

Consideraciones especiales Todo el software que no es objeto de prueba que es necesario y debe estar accesible. ¿Qué aplicaciones se usan normalmente? ¿Qué información se maneja en las aplicaciones que se usan normalmente, y que tamaño de información?.

5.1.9 Prueba de Instalación

La Prueba de Instalación tiene dos propósitos. Uno es asegurar que el software puede ser instalado en diferentes condiciones (como una nueva instalación, una actualización, y una instalación completa o personalizada) bajo condiciones normales y anormales.

Condiciones anormales pueden ser insuficiente espacio en disco, falta de privilegios para crear directorios, etc. El otro propósito es verificar que, una vez instalado, el software opera correctamente. Esto significa normalmente ejecutar un conjunto de pruebas que fueron desarrolladas para Prueba de Funcionalidad.

Justificación Evitar que surjan errores al momento de instalar la aplicación.

Objetivo de la prueba Verificar que el software objeto de prueba se instala correctamente en cada configuración de hardware requerida bajo las siguientes condiciones:

- instalación nueva, una nueva máquina, nunca instalada previamente con SICC
- actualización, máquina previamente instalada con SICC, con la misma versión
- actualización, máquina previamente instalada con SICC, con una versión anterior.

Técnica Manualmente o desarrollando programas, para validar la condición de la máquina destino (nueva, nunca instalado, misma versión, versión anterior ya instalada). Realizar la instalación, por último ejecutar un conjunto de pruebas funcionales ya implementadas para la Prueba de Funcionalidad.

Criterio de aceptación Las pruebas de funcionalidad de SICC se ejecutan exitosamente sin fallas.

Consideraciones especiales N/A.

5.1.10 Prueba de Documentos

Justificación La Prueba de Documentos es necesario para evitar inconsistencias, por lo que se debe asegurar que los documentos relacionados al software que se generen en el proceso sean correctos, consistentes y entendibles.

Objetivo de la prueba Verificar que el documento objeto de prueba sea: • Correcto, esto es, que cumpla con el formato y organización para el documento establecido en el proyecto. • Consistente, esto es, que el contenido del documento sea fiel a lo que hace referencia. Si el documento es Documentación de Usuario, que la explicación de un procedimiento sea exactamente como se realiza el procedimiento en el software, si se muestran pantallas que sean las correctas. • Entendible, esto es, que al leer el documento se entienda correctamente lo que expresa y sin ambigüedades, además que sea fácil de leer.

Técnica Para verificar que el documento es correcto se debe comparar con el estándar definido si existe o con las pautas de documentación y ver que el documento cumple con ellas.

Para verificar que el documento es Consistente se debe ejecutar el programa siguiendo el documento en caso de los Materiales de Soporte al Usuario y comprobar que lo que se explica en estos documentos es exactamente lo que se ejecuta en el programa. En caso de Documentación Técnica se debe revisar el código al cual corresponde la documentación y comprobar que dicha describe el código.

Para verificar que el documento es entendible, debe comprobar que se entiende correctamente, que no tiene ambigüedades y que sea fácil de leer.

Criterio de aceptación El documento expresa exactamente lo que debe expresar, no hay diferencias entre lo que está escrito y el objeto de la descripción (operación de software, código de programa, decisiones técnicas) y se entiende fácilmente.

Consideraciones especiales N/A.

6. Recursos

En esta sección se presentan los recursos recomendados para el proyecto SICC, sus principales responsabilidades y su conocimiento o habilidades.

6.1 Roles

En la tabla a continuación se muestra la composición de personal para el proyecto SICC en el área Verificación del Software.

Rol	Cantidad mínima de recursos recomendada	Responsabilidades
Responsable de verificación	1	Identifica, prioriza y diseña los casos de prueba. Genera el Plan de Verificación. Genera el Modelo de Prueba. Evalúa el esfuerzo necesario para verificar. Proporciona la dirección técnica. Adquiere los recursos apropiados. Proporciona informes sobre la verificación.
Asistente de verificación	3	Diseña casos de prueba. Ejecuta las pruebas. Registra los resultados de las pruebas. Documenta los pedidos de cambio.

7. Entregables

En esta sección se lista los documentos, herramientas e informes que se crearán, por quién, para quién. Para cada entregable deberá indicar las fechas en que son liberadas todas las versiones de este.

7.1 Modelo de Casos de Prueba

Documento	Modelo de Casos de Prueba
Creado por	El responsable de verificación, Sergio Cabrera
Para quién	Es la guía para realizar las pruebas del sistema y lo usarán los Asistentes de verificación y el responsable de verificación cuando se ejecuten las pruebas del sistema.
Fecha de liberación	Aun no definida

7.2 Informes de Verificación de sistema

Documento	Se genera un documento Informe de Verificación de Sistema por cada prueba de sistema que se realice.
Creado por	Equipo de testing.
Para quién	Es el retorno para los implementadores de la tarea de verificación, que detalla los errores encontrados para que puedan ser corregidos.
Fecha de liberación	Será liberado luego de cada verificación de sistema.

7.3 Informe final de verificación

Documento	El documento Informe final de verificación es el resumen de la verificación final del sistema antes de que sea liberado al entorno del usuario.
Creado por	El responsable de verificación, que toma como fuente de su trabajo los Informes de verificación.
Para quién	Indica el estado del sistema.
Fecha de liberación	Será liberado luego de la verificación final del sistema.

2.5 ESTÁNDARES, PRÁCTICAS Y CONVENCIONES

Esta sección establece los **estándares de calidad, prácticas y convenciones** que la organización y los equipos de proyecto deben seguir en la construcción del software. La adhesión a estándares de codificación y documentación uniformes mejora la mantenibilidad y legibilidad del software, reduce la probabilidad de errores y facilita las revisiones. Por ello, el SQAP define o referencia estándares específicos para las actividades de desarrollo, y también indica quién es responsable de verificar su cumplimiento.

2.5.1 ESTÁNDAR DE CODIFICACIÓN

Todo el código fuente desarrollado en iNeon debe seguir un Estándar de Codificación institucional. Este estándar define convenciones uniformes de estilo y estructura, que pueden incluir (entre otros) los siguientes aspectos:

- **Organización modular:** El software debe estructurarse en módulos o componentes independientes siguiendo el diseño establecido, promoviendo alta cohesión interna y bajo acoplamiento entre módulos.

- **Formato de código y nomenclatura:** Uso consistente de indentación, espaciado y llaves; **una sola instrucción por línea** para mejorar la legibilidad; nombres descriptivos para variables, constantes, funciones y clases que reflejen su función (por ejemplo, métodos con verbos y sustantivos significativos, y clases con nombres sustantivos). Se pueden emplear convenciones específicas, como notación *camelCase* o prefijos de tipo. En el estándar interno se indica, por ejemplo, que los nombres de funciones deben indicar claramente su funcionalidad, y los nombres de variables deben incluir una pista de su tipo o propósito.
- **Documentación en el código:** Cada módulo o archivo de código fuente debe iniciar con un bloque de cabecera que indique: nombre del módulo, autor, fecha de creación, historial de revisiones o cambios y un resumen de su propósito/funcionalidad. Asimismo, cada función o método público debe tener comentarios de documentación (por ejemplo usando una sintaxis estándar como Javadoc, XMLdoc, etc.) que expliquen qué hace, sus parámetros, valores devueltos y cualquier excepción que lance.
- **Manejo de errores y excepciones:** El código debe contemplar la captura y gestión adecuada de excepciones. Los mensajes de error o logs generados deben ser claros e indicar el módulo o ubicación donde ocurrió el problema para facilitar el debug.
- **Otras convenciones específicas:** Por ejemplo, si se programa en Java, se adoptarán las *Java Coding Conventions* originales de Sun/Oracle; si es en C#, las convenciones de Microsoft; etc. El estándar puede abarcar reglas sobre evitar construcciones de programación conocidas por ser propensas a error, pautas de optimización segura, etc.

2.5.2 ESTÁNDAR DE COMENTARIOS

Junto con el estilo de codificación, iNeon define un **estándar de comentarios en el código** que busca lograr un equilibrio entre código autoexplicativo y documentación embebida que aclare decisiones de diseño o partes complejas. Algunos lineamientos incluidos son:

- Los comentarios deben centrarse en explicar el **porqué** del código, más que describir literalmente el qué (que debería inferirse del propio código bien escrito). Es decir, usar comentarios para justificar algoritmos, explicar soluciones no evidentes o resaltar implicaciones importantes.

- Separar visiblemente los bloques de comentarios del código asociado (por ejemplo, mediante una línea en blanco antes y después, o sangría distinta) para mejorar la legibilidad.
- Utilizar comentarios de múltiples líneas (bloques) al inicio de módulos o antes de procedimientos complejos para describir la funcionalidad general, y comentarios de una sola línea para anotaciones breves o esclarecimientos puntuales dentro del código.
- Mantener los comentarios **actualizados**: cuando se modifica el código, se debe revisar y actualizar cualquier comentario que haya quedado desfasado. Los comentarios obsoletos son peligrosos porque desinforman.
- Evitar redundancia inútil: no comentar cada línea trivial (p.ej., `i = i + 1; // incrementar i en 1` es contraproducente). Los comentarios deben **agregar valor** más allá de lo obvio.
- Idioma: Los comentarios en el código deben escribirse en inglés, ya que es el estándar en la industria del software y facilita la colaboración con equipos internacionales y la comprensión por parte de desarrolladores de diferentes regiones. Solo se podrá utilizar otro idioma si todo el equipo lo acuerda explícitamente. En cualquier caso, se debe mantener la consistencia dentro de cada módulo.

2.5.3 RESPONSABLES DE VERIFICAR EL CUMPLIMIENTO

La adherencia a los estándares, prácticas y convenciones definidos (tanto de codificación, comentarios, como otros estándares de documentación o diseño) será verificada a lo largo del proyecto principalmente mediante:

- **Revisiones por pares (Peer Reviews):** Los propios desarrolladores, al hacer *code review* unos de otros (por ejemplo, antes de integrar cambios en la rama principal del repositorio), revisarán si el código cumple con las convenciones. iNeon instituye la política de que ningún código significativo se fusiona al producto sin al menos una revisión de otro desarrollador.
- **Auditorías SQA:** El grupo de Aseguramiento de Calidad realizará inspecciones formales en distintos puntos (por ejemplo, revisión formal de la especificación de requisitos, inspección de diseño, revisión técnica formal de código crítico). En cada una de estas actividades, SQA lleva **checklists** basadas en los estándares para verificar el cumplimiento. Por ejemplo, en una auditoría de código fuente, SQA seleccionará aleatoriamente módulos o clases y comprobará que siguen el estándar (nombrado de variables, estructura de comentarios, etc.). Asimismo, SQA

puede usar *herramientas de análisis estático* para detectar desviaciones automáticamente.

- **Responsables técnicos de área:** En algunos casos se designan roles líderes (por ejemplo, *Arquitecto de Software* o *Líder Técnico*) quienes son garantes de ciertos estándares. Un arquitecto podría ser responsable de revisar que las decisiones de diseño sigan la arquitectura aprobada; un líder de desarrollo podría revisar periódicamente los commits para guiar al equipo en la buena aplicación de estándares de codificación.

2.6 REVISIONES Y AUDITORÍAS

Las **revisiones y auditorías** son actividades de control de calidad fundamentales incluidas en el SQAP para evaluar tanto los productos de trabajo (documentos, código, etc.) como el proceso seguido, de manera independiente y objetiva. A diferencia de las pruebas (que validan el funcionamiento del software ejecutándolo), las revisiones se centran en *inspeccionar estáticamente* la calidad de entregables intermedios o finales, detectando defectos de forma, contenido, omisiones o desviaciones de estándares. Las auditorías, por su parte, verifican la adhesión al proceso establecido. IEEE 730 y 1028 recomiendan incluir en el plan distintos tipos de revisiones. En iNeon se contemplan principalmente **tres tipos de revisiones**:

Evaluación de la calidad de los productos

Este tipo de revisión se enfoca en examinar la calidad intrínseca de los productos de software generados en cada fase. Incluye, por ejemplo:

- **Revisiones de Requerimientos:** verificación del documento ERS (sección 2.4.1) asegurando que los requisitos estén correctamente especificados, trazables y comprobables. Se busca identificar requisitos inconsistentes, incompletos o erróneos antes de pasar a diseño.
- **Revisiones de Diseño:** inspecciones del SDD (sección 2.4.2) para chequear consistencia con los requisitos, buen uso de principios de diseño, y suficiente detalle. Se evalúa la calidad arquitectónica (modularidad, escalabilidad) y la corrección técnica del diseño.
- **Revisiones de Código (inspecciones estáticas):** análisis manual o asistido por herramientas del código fuente para encontrar defectos de lógica, violaciones de estándares de codificación, posibles errores de programación (ej.: manejo

incorrecto de excepciones, bucles infinitos), se examina también la complejidad y estructura del código.

- **Revisiones de Casos de Prueba:** asegurar que los casos de prueba diseñados cubren adecuadamente los requisitos (cobertura funcional) y escenarios relevantes, y que están correctamente definidos (pasos, datos esperados).
- **Revisión de Documentación de Usuario:** si aplica, revisar manuales de usuario, guías de instalación, etc., verificando que sean claros, completos y técnicamente exactos.

Revisar el ajuste al proceso

Mientras que las revisiones de producto evalúan la salida de cada fase, las **auditorías de proceso** se centran en verificar que el equipo está siguiendo el **proceso definido** en todos sus pasos. Un representante de SQA (auditor) revisa evidencias y observa prácticas para responder preguntas como:

- ¿Se están usando las plantillas y formatos requeridos para los documentos? (ej.: se utiliza la plantilla corporativa para la ERS, con todos sus apartados).
- ¿El equipo realizó las actividades que debía antes de avanzar? Por ejemplo, ¿hubo revisión de requisitos antes de iniciar el diseño?, ¿se ejecutaron pruebas unitarias de cada módulo antes de integrarlos?
- ¿Se están registrando adecuadamente los resultados? (ej.: existencia de registros de pruebas, listas de verificación llenadas en revisiones, actas de reuniones clave).
- ¿Los roles están cumpliendo sus responsabilidades? (ej.: el Líder Técnico aprobó el diseño, el Cliente validó los requisitos; el Comité de Control de Cambios está evaluando las solicitudes, etc.).
- ¿Se cumplen los criterios de control de cambios y configuración? (ver sección 2.7/2.10: comprobar que cada cambio de requisito está documentado y aprobado, que el repositorio de código muestra la estructura de ramas y etiquetado definidas, etc.).

Revisión Técnica Formal (RTF)

La Revisión Técnica Formal (RTF) es una forma rigurosa y estructurada de revisión, aplicada típicamente a productos de trabajo críticos (por ejemplo, la especificación de requisitos, el diseño arquitectónico, o código de módulos complejos). Se trata de un proceso documentado de inspección en el cual un equipo de revisión examina minuciosamente un producto en busca de *defectos o desviaciones* respecto a estándares

y requisitos. Las RTF actúan como un filtro que permite “purificar” los artefactos de ingeniería de software antes de avanzar en el ciclo de vida previniendo que errores mayores se propaguen.

Participantes y preparación: En una RTF típica participan:

- Un **Moderador** (puede ser alguien de SQA o un líder no autor del producto) que dirige la revisión.
- El **Autor** del producto (quien lo explica pero idealmente no dirige la reunión).
- Unos **2 a 4 Revisores** técnicos (desarrolladores, analistas o expertos externos al producto específico) encargados de examinar el producto detalladamente.
- A veces un **Secretario** que anota los hallazgos.

Previo a la reunión formal, el Moderador distribuye a los revisores el material a revisar (ej.: documento o código) junto con un **agenda** y las **instrucciones**. Todos deben prepararse leyendo el material individualmente y anotando posibles defectos. Esta etapa de preparación es vital para el éxito de la RTF, pues la efectividad depende de la revisión previa de cada participante.

Proceso de RTF: Una RTF suele dividirse en *fases estructuradas*:

- *Fase I – Inicial (Planificación):* El Moderador planifica la RTF, definiendo su **alcance** (qué documento o porción de código se revisará), los **objetivos** (por ejemplo, "verificar la corrección y completitud del módulo X"), selecciona al equipo revisor, fija la fecha, hora y lugar, y distribuye la documentación a examinar con suficiente antelación. También prepara una **agenda** indicando el orden del día: típicamente inicio (presentación del objetivo), revisión línea por línea o sección por sección del producto, discusión de hallazgos, y cierre. *Sólo se procede con la reunión formal si se cumplen ciertos **requerimientos mínimos**:* por ejemplo, que el artefacto a revisar esté *completo* y en un estado congelado, que todos los participantes clave confirmen su asistencia, y que hayan tenido tiempo suficiente para prepararse.
- *Fase II – Revisión individual:* (Generalmente ocurre antes de la reunión) Cada revisor analiza el producto por su cuenta, con la mente enfocada en su rol (algunos pueden buscar errores de lógica, otros verificar cumplimiento de estándares, etc.). Anotan los posibles defectos o dudas encontradas.
- *Fase III – Reunión de revisión:* Se realiza la reunión presencial (o virtual) con todos los participantes. El autor suele comenzar dando una visión general del producto. Luego, de forma sistemática se discuten los puntos que los revisores prepararon. **Importante:** la RTF no es para solucionar los defectos en vivo, sino para

identificarlos claramente. El Moderador se asegura de mantener la reunión enfocada y productiva. Cada hallazgo se clasifica (e.g., defectos mayores, menores, sugerencias) y se anota en el **Informe de Revisión Técnica**. Se revisa el producto, no al productor – las críticas se dirigen al documento/código, evitando personalizaciones.

- *Fase IV – Reporte de resultados:* Tras la reunión, el Secretario (o Moderador) elabora el **Informe de RTF**, que resume los hallazgos (lista de defectos con su severidad y ubicación), las acciones acordadas y quién las realizará. Este informe se distribuye al autor y al Gerente de Proyecto. Se podría decidir en la reunión un resultado global, por ejemplo: "Aprobado con condiciones" (debe corregirse X e Y y luego se considerará aprobado sin otra reunión).
- *Fase V – Acciones correctivas y seguimiento:* El autor corrige los defectos identificados. SQA/Moderador verifica posteriormente que todas las acciones correctivas se completaron adecuadamente (**seguimiento**). Si los defectos eran críticos, puede incluso planificarse una mini-revisión de verificación de las correcciones, o repetir la RTF si hubo cambios muy grandes.

Elementos mínimos a revisar: Como política institucional, se exige que al menos los siguientes artefactos pasen por una RTF (u otro tipo de revisión formal equivalente) en cada proyecto:

- La **Especificación de Requisitos de Software** completa.
- El **Diseño de alto nivel** (arquitectura del sistema) y, de ser posible, también partes del diseño detallado críticas o complejas.
- Fragmentos de **Código fuente** de módulos de alta criticidad (p.ej., módulos de seguridad, algoritmos complejos, o de los que dependan muchos otros componentes) – en caso de no poder revisar todo el código, se prioriza por criticidad y complejidad.
- El **Plan de Pruebas** o casos de prueba principales, para asegurar que cubren los requisitos y están bien diseñados.
- Cualquier otro producto cuya calidad sea vital (por ejemplo, manual de usuario en un sistema donde la usabilidad sea crítica).

Este conjunto mínimo garantiza que los puntos neurálgicos del proyecto reciban escrutinio formal.

Agenda de una RTF típica: la agenda, preparada en la fase inicial, incluirá: introducción (objetivo de la revisión), lista de documentos a revisar y secciones asignadas a cada revisor, duración prevista (las RTF suelen limitarse a 2 horas para mantener efectividad),

reglas (por ejemplo, apagar teléfonos, mantener espíritu constructivo), y cierre (resumen de hallazgos y acuerdos). Cada participante recibe la agenda junto con el material a revisar, de modo que llega a la reunión conociendo el plan.

2.7 GESTIÓN DE CONFIGURACIÓN

La **Gestión de Configuración de Software (SCM)** es esencial para mantener la integridad y trazabilidad de los productos del proyecto a lo largo del tiempo. Abarca el control de versiones, la identificación de ítems de configuración, el control de cambios y la auditoría de las configuraciones. El rol del aseguramiento de calidad en esta área es verificar que las prácticas de configuración establecidas se lleven a cabo correctamente y salvaguardar que el producto entregado corresponde fielmente a lo desarrollado y probado.

En iNeon existe un **Plan de Gestión de Configuración** separado (SCMP), pero el SQAP resume los puntos clave y define actividades SQA de monitoreo. El objetivo del SQA en esta área es asegurar que se realizan las actividades de configuración definidas y según lo establecido en el proceso. Algunas acciones mínimas que el grupo SQA realizará respecto a SCM son:

- **Control de línea base:** Verificar que se genera la **Línea Base** del proyecto en los momentos estipulados. Además, SQA comprueba que cada línea base generada es **correcta y completa** (contiene todos los componentes previstos, en las versiones acordadas, y está etiquetada adecuadamente).
- **Registro de cambios:** Confirmar que el responsable de SCM mantiene un registro completo de los cambios realizados, incluyendo: cambios de requerimientos (con su trazabilidad a versiones de documentos), cambios de diseño, modificaciones de código, resultados de verificación, actualizaciones de documentación, etc. Esto suele lograrse mediante herramientas de control de versiones (p. ej. Git) y sistemas de *tracking* para requerimientos y tareas. SQA puede auditar el repositorio para ver historial de commits, etiquetados de release, y comparar con las políticas definidas (por ejemplo, convención de nombres de ramas y tags).
- **Procedimientos del Control de Cambios:** Monitorear que el Comité de Control de Cambios (CCB) o el proceso definido para aprobar cambios esté funcionando efectivamente. Por ejemplo, SQA revisará que toda Solicitud de Cambio mayor tenga la documentación pertinente (formulario de cambio con descripción, análisis de impacto) y evidencias de su evaluación/aprobación antes de implementarse. Si hay cambios realizados sin autorización, es una no conformidad grave. SQA puede

asistir a reuniones del CCB como observador para constatar que se siguen las pautas del SCMP.

- **Integridad de las configuraciones:** A intervalos, SQA realiza auditorías de configuración, que implican seleccionar una versión declarada (por ejemplo, "Release 1.0") y comprobar que todos sus componentes correspondientes están presentes y correctamente identificados. Se revisan también los elementos de soporte: que la documentación entregable coincida con la versión del software, que los manuales refieran a la versión correcta, etc.
- **Herramientas de SCM:** Asegurar que se utilizan las herramientas acordadas (por ejemplo, repositorio Git central de la empresa, sistema de gestión de incidencias JIRA, etc.) y que el equipo conoce su uso. SQA puede verificar que los desarrolladores realizan commits atómicos y descriptivos, que utilizan ramas según la estrategia definida (ej. branching model GitFlow u otro), y que las *builds* son reproducibles a partir de la configuración almacenada.

2.8 GESTIÓN DE PROBLEMAS Y ACCIONES CORRELATIVAS

Durante el ciclo de vida de desarrollo es normal que surjan problemas o incidencias: defectos del software, discrepancias en documentos, fallos en pruebas, etc. La gestión de problemas y acciones correctivas establece cómo se reportan, analizan y solucionan estos problemas, asegurando que *no queden olvidados* y que su resolución sea verificada. Un proceso riguroso de gestión de incidencias es fundamental para mejorar la calidad, ya que provee la retroalimentación y seguimiento necesario ante los hallazgos de verificación y validación.

El propósito de un sistema formal de gestión de problemas es:

- **Documentar todos los problemas** encontrados en el producto (requisitos, código, prueba, etc.), realizar su seguimiento hasta su corrección y evitar que sean ignorados o se repitan.
- **Evaluar la validez** de cada reporte de problema (evitando falsos positivos o duplicados) y priorizar su severidad e impacto.
- **Retroalimentar al desarrollador y al usuario** sobre el estado de cada problema reportado, manteniendo transparencia. Por ejemplo, informar cuando un bug ha sido identificado, cuando está en corrección y cuando se resuelve en una nueva versión.
- **Proporcionar datos para métricas:** los datos de problemas sirven para medir la calidad del software (por ejemplo, densidad de defectos, tiempo de respuesta a

incidencias) y predecir la fiabilidad futura del producto. También permiten análisis de causa raíz para mejorar el proceso (identificando áreas débiles del desarrollo).

En iNeon, se emplea una herramienta de *tracking* de incidencias (por ejemplo, JIRA, Redmine u otra) para registrar y gestionar todos los problemas. El SQAP define los siguientes aspectos del proceso:

- **Reporte de problemas:** Todo miembro del equipo (o usuario de pruebas) que detecte un defecto o no conformidad debe llenar un **Informe de Problema** en la herramienta designada. Este reporte incluirá al menos:
 - Fecha y versión en que se encontró el problema.
 - Identificación única (ID) del problema.
 - Descripción detallada del comportamiento observado vs el esperado (en caso de bug de software) o de la discrepancia (en caso de incidencia de documento/proceso).
 - Paso a paso para reproducir el problema, si aplica.
 - Severidad o impacto (por ejemplo: crítica, mayor, menor) y prioridad (urgente, alta, media, baja).
 - Módulo o componente involucrado.
 - Persona que reporta (quien detectó) y firma o validación de SQA si corresponde.

Este reporte, en formato electrónico, queda en estado "Abierto/Pendiente" y es asignado a un responsable (típicamente el desarrollador del módulo o el líder técnico). El grupo SQA supervisa la cola de problemas para asegurarse de que ninguno queda sin asignar o sin atender.

- **Seguimiento y resolución:** La organización responsable de gestionar los problemas es la de SQA, comandada en conjunto con el líder del proyecto. Esto implica que SQA:
 - **Clasifica y valida** los nuevos problemas reportados (asegurando que estén bien documentados y reproducidos; puede descartarse o fusionarse duplicados).
 - **Convoca reuniones de análisis** de ser necesarias (por ejemplo, para problemas críticos SQA puede organizar una reunión con desarrollador, tester y posiblemente el cliente, para evaluar impacto y plan de acción).
 - **Determina el plan de acción correctiva:** en coordinación con el gerente de proyecto, deciden cuándo y cómo se solventará el problema

(inmediatamente, en la siguiente iteración, con qué recursos). Esto puede involucrar replanificación si es un cambio mayor.

- **Registra la solución** propuesta y ejecutada: una vez que el desarrollador corrige el defecto, registra en el sistema la solución (p. ej., "Corregido en versión X. Utilizado algoritmo Y en lugar de Z para evitar desbordamiento").
- **Cierra el problema** solo después de verificar la efectividad de la corrección (SQA o el reportero realizan pruebas de revalidación). Se documenta la fecha de cierre y la versión en que quedó resuelto.

Periódicamente, SQA puede generar un **Reporte Resumen de Problemas** para la gerencia, indicando cuántos problemas se han abierto, cerrados, pendientes, tiempos promedio de resolución, etc., para evaluar la salud del proyecto.

Cabe señalar que el tiempo de respuesta a problemas críticos debe ser acotado. Si un problema bloqueante no es atendido en un plazo definido (por ejemplo 24-48 horas), SQA escalará el asunto a la dirección del proyecto para acción inmediata.

2.9 HERRAMIENTAS, TÉCNICAS Y METODOLOGÍAS

En apoyo a todos los procesos descritos, iNeon emplea diversas herramientas de software, técnicas de ingeniería y metodologías que facilitan y potencian las actividades de calidad. Este apartado identifica las principales:

- **Herramientas de Control de Versiones:** Se utiliza un sistema de control de versiones (por ejemplo, **Git** con un repositorio central GitLab/GitHub) para gestionar el código fuente y otros artefactos versionables. Esto permite llevar el historial de cambios, crear ramas para desarrollo paralelo y garantizar integridad de las versiones (ver sección 2.10). SQA puede configurar reglas en el repositorio (como requerir *pull requests* con aprobación antes de fusionar código, o ejecutar pipelines de CI con pruebas).
- **Herramientas de Gestión de Requisitos:** Un software de administración de requerimientos (desde documentos estructurados en Confluence/Office hasta suites especializadas) puede emplearse para enumerar cada requisito con su identificador y descripción. Así se facilita la trazabilidad (vincular requisitos con casos de prueba, commits, etc.).
- **Herramientas de Seguimiento de Incidencias y Tareas:** Como mencionado, sistemas tipo **JIRA**, **Trello** o **Redmine** se utilizan para registrar bugs, tareas de desarrollo y mejoras. SQA configura flujos de trabajo en estas herramientas para

asegurar que cada incidencia pase por estados de *Abierto-En curso-Resuelto-Verificado-Cerrado*, con responsables asignados.

- **Herramientas de Integración Continua (CI):** iNeon busca automatizar la compilación y pruebas. Por ejemplo, mediante **Jenkins, GitLab CI/CD o GitHub Actions**, al hacer commit se construye el software automáticamente y se ejecutan suites de pruebas unitarias y quizás pruebas estáticas de calidad de código. Esto ayuda a detectar rápidamente regresiones o incumplimiento de estándares (si se integran linters, analizadores estáticos como SonarQube).
- **Técnicas de Revisión y Análisis Estático:** Además de las revisiones manuales, se usan herramientas de *análisis estático de código* para complementar: linters (como ESLint para JavaScript, Pylint para Python, Checkstyle para Java) que detectan violaciones de estilo automáticamente; análisis de complejidad; detectores de duplicación de código; etc. Estas técnicas permiten encontrar ciertos defectos sin ejecutar el programa.
- **Técnicas de Prueba:** El equipo aplica un enfoque de pruebas robusto:
 - **Pruebas unitarias automatizadas:** cada módulo clave tiene un conjunto de unit tests (usando frameworks como JUnit, NUnit, pytest, etc.), promoviendo la cobertura de código.
 - **Pruebas de integración y de sistema:** se preparan datos de prueba y entornos controlados para simular escenarios integrales. Se usan técnicas como *tests de caja negra* (basados en requisitos) y *caja blanca* (basados en la estructura) según corresponda.
 - **Pruebas de rendimiento y de seguridad:** en proyectos que lo requieran, se usan herramientas de *stress test* (p. ej., JMeter) o escáneres de seguridad (OWASP ZAP, SonarQube plugins) para validar estos atributos de calidad.
- **Metodología de desarrollo adoptada:** Como se indicó en 2.1.3, se sigue un modelo V (predictivo). No obstante, iNeon incorpora prácticas ágiles cuando es posible sin comprometer la documentación de calidad. Por ejemplo, aunque el ciclo global es en V, internamente los desarrolladores pueden iterar con entregas incrementales. Lo importante es que *en cada incremento* se aplican las actividades SQA definidas (revisiones, pruebas). En definitiva, la metodología en V se ve apoyada por las herramientas mencionadas (CI, tracking) para agilizar la retroalimentación.
- **Estándares técnicos de referencia:** Más allá de los estándares internos, iNeon se apoya en estándares IEEE/ISO reconocidos como guías técnicas para asegurar la calidad en cada disciplina. Como ya se listó en 2.2, por ejemplo:
 - IEEE 830/29148 para escribir buenos documentos de requisitos.

- IEEE 1016 para descripciones de diseño claras.
- IEEE 1008 para la realización sistemática de pruebas unitarias.
- IEEE 1063 para estructurar la documentación de usuario.
- IEEE 1028 para la ejecución correcta de revisiones y auditorías de software.

2.10 CONTROL DE CÓDIGO

El **Control de Código** se refiere al conjunto de métodos y procedimientos para gestionar el almacenamiento, la versión y la integridad del código fuente del software durante el proyecto. Un control de código riguroso asegura que:

- Se sabe **qué software (qué ficheros y contenido)** se va a controlar bajo configuración.
- Existe un método estándar para **identificar, etiquetar y catalogar** cada componente del software. Esto suele implicar definir convenciones de nombres (p.ej., nomenclatura de archivos y directorios), usar identificadores de versión y metadatos en el repositorio.
- Se lista la **localización física** del software bajo control. Es decir, se conoce dónde reside el repositorio maestro (p.ej., en un servidor Git central, con copias locales en las máquinas de desarrolladores). También se sabe qué ramas de código existen (p.ej., *main*, *develop*, *release-x*, etc.) y su propósito.
- Se describen los procedimientos para **realizar copias de seguridad** del código y para **restaurarlo** en caso necesario.
- Se establecen los procedimientos para la **distribución de copias** del software. Esto aplica especialmente cuando se lanza una versión: cómo se genera el *build* instalable o ejecutable, quién está autorizado a hacerlo y cómo se entregan los paquetes (por ejemplo, firmado digitalmente, subido a repositorio de releases).
- Se identifican qué **documentos asociados** se ven afectados por cambios de código.
- Se describen los pasos para la **construcción de una nueva versión** del software. Esto incluye detallar el proceso de *build*: comandos de compilación, herramientas involucradas, dependencias externas, etc., idealmente automatizado por scripts de CI. Un nuevo miembro del equipo, siguiendo este procedimiento, debería poder generar el producto exactamente como se hace oficialmente.

El SQAP requiere que exista un Procedimiento de Control de Código documentado con los puntos anteriores. SQA se asegurará que dicho procedimiento esté en vigor y que el equipo lo siga. Por ejemplo, algunas políticas concretas de iNeon podrían ser:

- Todo desarrollo ocurre en ramas de característica; para fusionar a la rama principal se requiere aprobación (código revisado).
- Se etiquetará cada versión liberable con un tag en Git (vX.Y.Z) y se generará un *release note* enumerando cambios.
- No se permitirá código fuera de control: cualquier archivo fuente debe residir en el repositorio, evitar intercambios de código por vías informales (correo, chat) sin registro.
- En cuanto a herramientas, utilizar un **Sistema de Control de Versiones Distribuido** (Git) con un flujo definido (p.ej., GitFlow). Para proyectos que lo requieran, quizá *ganchos (hooks) de commit* para verificar formato de código o ejecución de pruebas unitarias antes de aceptar commits.

Verificación SQA: El grupo SQA realizará auditorías específicas al control de código. Por ejemplo, antes de una release, SQA verificará que:

- La etiqueta de versión en Git corresponde a la versión documentada en las notas de entrega y en el ejecutable.
- No hay diferencias entre el código compilado y el código en repositorio (integridad de build).
- Los desarrolladores no tienen ramas locales no incorporadas si estas afectaban requisitos del release.
- Las copias de seguridad se realizaron (se puede pedir restaurar una backup de prueba para verificar su utilidad).
- Los accesos al repositorio están restringidos a personal autorizado (esto vincula con control de medios en 2.11 en cuanto a acceso).

2.11 CONTROL DE MEDIOS

El Control de Medios se enfoca en proteger los medios físicos o lógicos donde reside el software y su documentación, contra accesos no autorizados, daños o degradaciones. En el contexto actual, "medios" se refiere principalmente a soportes de almacenamiento y distribución, por ejemplo: servidores, discos duros, repositorios en la nube, dispositivos extraíbles, etc., que contienen el código fuente, ejecutables, documentación y backups del proyecto.

Las medidas de control de medios en iNeon incluyen:

- **Seguridad de acceso:** Sólo personal autorizado debe poder acceder a los repositorios de código y a los archivos de configuración del proyecto. Para ello:

- Los repositorios están protegidos con credenciales y posiblemente autenticación de dos factores. Cada desarrollador tiene su usuario individual; cuando alguien deja el equipo, se revocan sus accesos inmediatamente.
- Los servidores de integración continua, bases de datos de prueba y demás entornos también están bajo control de acceso. Se aplican perfiles de permisos (por ej., desarrolladores pueden leer/escribir código; testers quizás solo lectura; clientes externos no tienen acceso directo, etc.).
- **Prevención de daños y degradación:** Asegurar que los datos del proyecto se almacenan en entornos estables y con **redundancia**. Los servidores físicos están en ubicación adecuada (data center con UPS, control de clima), o se usa infraestructura cloud con garantías de disponibilidad. Además, se realizan copias de seguridad periódicas (como mencionado en 2.10) para evitar pérdida de información por fallas. SQA comprueba que se sigan las políticas de backup (frecuencia, almacenamiento externo).
- **Control del entorno de almacenamiento:** Evitar que factores ambientales o técnicos comprometan los medios:
 - En sistemas de archivos compartidos, definir controles para que virus o malware no infecten los archivos fuente (uso de antivirus actualizado).
 - Monitoreo de integridad: se podría implementar hashes o sumas de verificación de archivos críticos para detectar corrupción.
 - Renovación/rotación de soportes si fueran físicos (por ejemplo, si se usan cintas o discos externos para backup, seguir ciclos de renovación para prevenir deterioro).
- **Identificación y etiquetado de medios:** Cualquier medio físico (p.ej., un disco externo con una entrega) debe estar **etiquetado claramente** con su contenido, versión y fecha. Así se evitan confusiones o uso accidental de medios equivocados.
- **Distribución controlada de copias:** Cuando se entregan versiones al cliente en algún medio (por ejemplo, un instalador en USB o un paquete vía repositorio), se sigue un procedimiento:
 - Registrar qué se entrega, a quién, cuándo.
 - Asegurarse de que el medio esté libre de malware (ejecutar un *checksum* y análisis antivirus antes de enviar).
 - Proteger el medio contra alteraciones: si es digital, mediante firma digital o envío por canal seguro; si es físico, con sellos o empaques inviolables si aplica.
 - Evitar copias no autorizadas: desalentar la reproducción no controlada del software entregado, a través de acuerdos de confidencialidad y licencias.

- **Conservación de documentación física:** Aunque la mayor parte es digital, si hubiera documentación en papel relevante (contratos firmados, diagramas dibujados a mano que luego se escanean, etc.), ésta se guarda en archivos bajo llave. Sin embargo, la política es digitalizar todo documento para integrarlo en los repositorios controlados.

2.12 CONTROL DE SUMINISTROS Y SUBCONTRATOS

A veces, parte del software o servicios asociados pueden provenir de proveedores externos o ser desarrollados por terceros subcontratados. Esto puede incluir la compra de componentes de software (librerías, frameworks), uso de módulos de código abierto, o contratación de empresas/consultores para desarrollar algún subsistema. El Control de Suministros y Subcontratos busca garantizar que cualquier software o componente proporcionado por terceros cumpla con los requisitos de calidad y técnicos del proyecto, integrándose sin problemas al producto final.

Las prácticas de iNeon en este sentido incluyen:

- **Evaluación de proveedores:** Antes de adquirir o subcontratar software, se evalúa la reputación y calidad del proveedor. Por ejemplo, si se considera usar una librería de terceros, SQA y el equipo técnico revisan su madurez (versión, actualizaciones frecuentes), licencia, documentación, y la comunidad que la respalda. Si es un proveedor comercial, se revisan sus certificaciones de calidad (¿tienen ISO 9001? ¿CMMI?), casos de éxito y referencias.
- **Especificaciones en contratos:** En contratos con subcontratistas, iNeon incluye cláusulas que establecen criterios de calidad. Por ejemplo, definir estándares que deben seguir, entregables intermedios, derecho de inspección por parte de iNeon, métricas de rendimiento a cumplir, etc. Incluso se puede requerir que el subcontratista implemente un plan SQA propio alineado con IEEE 730.
- **Revisión de entregables externos:** Todo entregable recibido de un proveedor (sea código fuente, binario, documentación) pasa por una **verificación interna** antes de aceptarlo:
 - Si es código fuente: se somete a análisis estático y pruebas (idealmente con los mismos criterios que el código interno). También se revisa que esté bien documentado para futuras mantenciones.
 - Si es un producto software adquirido: SQA asegura que se realicen pruebas de integración para ver cómo interactúa con nuestro sistema, y pruebas de requisitos para ver si cumple lo prometido.

- Si es documentación o diseños: se inspecciona que cubran lo esperado y no presenten inconsistencias.
- **Cumplimiento de requisitos técnicos:** Se establecen pruebas de aceptación específicas para componentes de terceros. Por ejemplo, si se compra un módulo de autenticación, se prueba que cumple con los requisitos de seguridad definidos. El estándar aquí es que el software de terceros debe cumplir con los mismos requisitos que se impondrían a uno desarrollado internamente.
- **Gestión de incidencias con proveedores:** Si se encuentran defectos en componentes externos, iNeon los reporta formalmente al proveedor y hace seguimiento de su corrección (similar al proceso interno, pero hacia afuera). En casos críticos, si el proveedor no responde a tiempo, iNeon podría decidir implementar correcciones provisionales o cambiar de proveedor/componente.
- **Control de versiones de componentes externos:** Se lleva un registro de qué versión exacta de componentes de terceros se está usando. Si hay actualizaciones, se evalúan antes de incorporarlas (no actualizar automáticamente sin pruebas). Esto evita introducir regresiones de terceros inadvertidamente.
- **Control de proveedores múltiples:** Si hay varios subcontratistas trabajando en distintas partes, SQA puede organizar *reuniones conjuntas de calidad* para alinear criterios y detectar a tiempo posibles incompatibilidades en interfaces entre lo que desarrolla cada uno.

2.13 RECOLECCIÓN, MANTENIMIENTO Y RETENCIÓN DE REGISTROS

Un principio clave de los sistemas de calidad es: "Lo que no se registra, no se puede probar ni mejorar". Por ello, el SQAP define qué registros de calidad deben generarse durante el proyecto, cómo se mantienen y por cuánto tiempo se retienen.

Identificación de documentación a retener: Se identifican todos aquellos documentos y datos que deben conservarse a largo plazo. Por ejemplo:

- **Plan de Proyecto y Plan de Calidad (SQAP):** versiones aprobadas y cualquier actualización, como referencia histórica.
- **Especificación de Requisitos y de Diseño baselizadas:** representan qué se acordó construir, cruciales para mantenimientos futuros y para resolver posibles disputas sobre alcance.
- **Casos de prueba y resultados de pruebas:** especialmente los de aceptación y sistema. Incluir evidencias de ejecución (logs, capturas de pantalla, informes de herramientas). Esto demuestra que se cumplió el procedimiento de validación.

- **Informes de revisiones y auditorías:** actas de RTF, listas de verificación llenas, auditorías de proceso, etc. Son prueba de que se efectuaron y qué hallaron.
- **Registros de cambios y controles de configuración:** lista de versiones liberadas, solicitudes de cambio aprobadas, etc., para trazar la evolución del software.
- **Reporte final de calidad/SQA:** un informe resumen elaborado al cierre del proyecto por SQA, que indique el nivel de cumplimiento de calidad obtenido, estadísticas de defectos, y sign-off final.
- **Correspondencia formal con el cliente relacionada a calidad:** por ejemplo, aprobaciones formales de entregables, aceptaciones de producto, acuerdos de cómo tratar ciertos defectos pos-entrega, etc.

Métodos y facilidades de recolección y mantenimiento: Todos los registros mencionados se recopilan de manera continua:

- Muchos se generan automáticamente por las herramientas (p.ej., el sistema de seguimiento de problemas puede exportar todos los tickets cerrados; la herramienta de CI guarda logs de compilación).
- Otros son documentos manuales (minutas, checklists) que SQA digitaliza si originalmente fueron en papel y archiva en el repositorio de configuración.
- Se define una **estructura de almacenamiento** para estos registros, por ejemplo un espacio en el servidor de proyectos dedicado a "Archivos de calidad". Allí SQA organiza carpetas por proyecto y por tipo de registro.
- Durante el proyecto, estos registros están *vivos* y se actualizan; SQA es responsable de mantenerlos actualizados (por ejemplo, agregar la última acta de revisión, el último resultado de pruebas, etc., al repositorio correspondiente).

Retención de registros: El plan especifica por cuánto tiempo deben conservarse cada tipo de registro:

- Por política general de iNeon, se retienen los registros completos de cada proyecto **al menos durante 3 años** tras su finalización (a menos que normas legales exijan más). Esto cubre el periodo en que suelen ocurrir garantías o mantenimientos iniciales.
- Algunos registros críticos (como contratos, especificaciones de requisitos, código fuente) se pueden conservar incluso indefinidamente, dado que pueden ser útiles para mantenimientos futuros mucho tiempo después.

- Si la empresa cuenta con certificaciones (ISO, etc.), suele requerir retener cierta documentación como evidencia de cumplimiento para auditorías, por lo que se alinea con esos requisitos.
- La retención incluye mantener accesibles los medios: por ejemplo, asegurarse de que los formatos de archivo usados sigan siendo legibles en el futuro (migrar documentos a formatos estándar si estaban en herramientas propietarias).
- Cuando expira el periodo de retención, los registros pueden ser archivados fuera de línea o destruidos de forma segura, según la política corporativa. Esto se hace con aprobación del Gerente de Calidad.

Responsabilidades: La organización responsable de estas tareas es la de SQA en coordinación con quien tenga la custodia de documentos (p. ej., un Administrador de Configuración). SQA debe asegurarse de que se identificó toda documentación a retener y de que se están recolectando adecuadamente las evidencias durante el proyecto.